AI Engineer Training Task Bank

Duration: 6 weeks

How to read this Task Bank

One main project per week

Each week centers on a single **main project** — one substantial build — plus a shorter piece of **background** (a written explainer or hands-on check). This is intentional: chaining skills under one goal teaches how they interact, which is the real job. You will switch contexts less and build something you can actually show at the end of each week.

Every task gives you

- **What you're doing —** the assignment in one or two plain sentences.
- **Background —** the minimum you need to understand before coding.
- **Steps —** the concrete path, in order.
- **Worked example / expected output —** what a correct result looks like.
- **Target —** a measurable bar — this is "Meets" on the rubric.
- **Deliverable —** the exact file(s) to produce.
- **Rubric —** four levels, so grading is consistent, not subjective.

Hint boxes are help, not extra work

Throughout, you'll see **Hint** boxes. These explain a confusing term or suggest how to approach a hard step in plain language. They are **optional support**, never additional requirements — if a concept already makes sense to you, skip the hint. They exist so you're never stuck on jargon or a blank page.

Notebook vs. script — and why it matters

- **Jupyter notebook (.ipynb)** — real, runnable Python that runs in cells with output shown inline. Used *only* for exploratory and analysis work (loading data, plotting, trying things), where seeing results step-by-step is the right tool.
- **Script / module (.py, .js)** — used for anything that *is a program that runs*: a pipeline, an API integration, a chatbot, an agent. These must be proper runnable files, not notebooks.

Each task says which it wants and why.

Language / stack policy

Tasks are language-agnostic unless marked **[Python required]** or **[Python recommended]**. Some AI tooling only works well in Python; those tasks must be done in Python, because every AI engineer needs Python regardless of their main stack.

Effort labels

● ~1 day ●● 2–3 days ●●● ~1 week

Each week's **core** tasks fit inside five working days. Tasks marked **Stretch** are optional and only attempted if the week is going smoothly.

How grading works (read once, applies everywhere)

Every main project is scored on a four-level rubric. The levels are the same program-wide so that two different mentors grade the same work the same way:

Not yet	Approaching	Meets (target)	Exceeds
Key requirements are missing or the result does not run / does not meet the basic goal.	It runs and is close, but misses the target, skips a required piece, or the trainee can't yet explain it.	Hits the stated numeric target, includes every required piece, and the trainee can explain what they did and why.	Clearly beyond target: deeper analysis, cleaner code, extra rigor, or insight the task didn't demand.

The bar to advance is "Meets"

"Meets" always corresponds to the task's **Target** line. Crucially, "Meets" requires **understanding, not just a passing number** — a trainee who copied a result but cannot explain it has not met the bar. "Exceeds" is never required; it simply recognizes work that goes beyond. A mentor reviews and signs off each week before the next begins.

A note on the three threads. Watch for the ✨ **Thread** markers. They flag where the **evaluation mindset**, the **cosine-similarity** idea, and **professional hygiene** carry forward across weeks. Noticing them is how the program connects into one picture instead of six.

Week 1 — Foundations, Environment & Data Fluency

Week 1 does three things: sets up a professional development environment you'll use all program long, builds the minimum math intuition that makes everything later make sense, and — the heart of the week — teaches you to load, clean, explore, and reshape **real, messy data**, because in AI the data work comes before any model. Nothing this week needs prior AI knowledge. A solid Week 1 makes Weeks 2–6 far easier.

> **This week's main project**
>
> One real, messy dataset, carried end-to-end in a **single reproducible pipeline**: set up your tools and repo, then **load → clean → explore → engineer features**, finishing with a short findings note. A from-scratch math primer rides along as the concept companion — and plants the cosine-similarity seed you'll harvest in Weeks 3 and 5.

Tools (all free): Python 3.11+, venv (or conda), Jupyter, Git/GitHub, pandas, NumPy, matplotlib/seaborn.

Task 1.1 — Professional setup: Python, virtual env, Jupyter, Git

●● 2–3 days [Python required]

What you're doing

Install Python the right way, prove you can create **isolated** project environments, and put your work under version control. In real work, different projects need different package versions and committing secrets can cost real money — this task builds the habits that prevent both.

> **Hint — what a "virtual environment" actually is**
>
> Think of it as a clean, private toolbox for one project. Without it, every Python package you install goes into one shared pile, and two projects that need different versions of the same tool will clash. A virtual environment gives this project its *own* toolbox, separate from everything else on your computer. You "activate" it to step into that toolbox; when your terminal prompt shows (ai-training) at the start of the line, you're inside it.

Steps

1. Install Python 3.11+. Create and activate a virtual environment named ai-training (built-in venv is recommended; conda is fine). Confirm you're "inside" it (prompt changes; which python points into the env).
2. Install numpy, pandas, matplotlib, seaborn, jupyter. Freeze them: pip freeze > requirements.txt.
3. Launch JupyterLab, create a notebook, import the libraries and print each version.
4. Install and configure Git. Create a private repo ai-traineeship and clone it. Add a .gitignore that excludes venv/, .env, __pycache__/.

5. Make at least 5 commits with clear messages as you work through the week (e.g. “Add cleaning pipeline”, not “stuff”).

Hint — Git in one breath

Git is a **save-history for your code**. A **commit** is one saved snapshot with a short note describing what changed; **push** uploads your snapshots to GitHub (the website) so they’re backed up and shareable. The .gitignore file is a list of things to *never* save — like your secret keys and your virtual-environment folder. If you’re new to it, do the free “Introduction to GitHub” guide once and you’ll have enough to work all program long.

Hard rules — professional hygiene ✨ Thread

- Never run sudo pip install or install into system Python. If you typed sudo to install a package, stop — you’ve done it wrong.
- **Never commit secrets or your venv folder.** That is what .gitignore is for. This rule holds in every week from here on.

Target

A teammate can take your requirements.txt, run pip install -r requirements.txt in a fresh environment, and reproduce your setup with zero errors; your repo is online with a clean history and no secrets.

Deliverable

requirements.txt + a setup_check.ipynb that prints library versions + a link to your GitHub repo.

Task 1.2 — Explain the core math, then implement it from scratch

●● 2–3 days [Python recommended for Part B]

What you're doing

Build just enough math intuition to *read and reason about* ML — not to pass an exam. Two parts: explain the ideas in your own words, then implement them in code and check against NumPy.

Part A — Explainer (~1.5 pages, your own words)

Learn first from free resources: 3Blue1Brown's *Essence of Linear Algebra* and *Essence of Calculus*; Khan Academy for probability. Then cover each with a 2–4 sentence explanation and a tiny worked example:

- **Vectors & matrices** — what they are; do one 2×2 matrix multiplication by hand.
- **Dot product** — compute [1,2,3]·[4,5,6] = 32 by hand; explain what it tells you (how aligned two vectors are).
- **Derivative / gradient** — what a derivative represents (slope/rate of change) and what a gradient is (direction of steepest increase). This is the heart of how models learn.
- **Mean, variance, standard deviation** — compute all three for [2,4,4,4,5,5,7,9] by hand.
- **Probability & Bayes** — explain conditional probability, then solve: a disease affects 1% of people; a test is 99% accurate; you test positive — what's the real probability you're sick? (The surprising ~50% answer is the point.)

> **Hint — how to think about the disease-test puzzle**
>
> The trick is to imagine a real crowd instead of juggling percentages. Picture **10,000 people**. 1% are truly sick → 100 sick, 9,900 healthy. The test catches 99% of the 100 sick → about 99 true positives. But it also wrongly flags 1% of the 9,900 healthy → about 99 false positives. So of everyone who tests positive (99 + 99 ≈ 198), only about half are actually sick. That's why the answer is ~50%, not 99% — because the healthy group is so much larger that its few mistakes match the sick group's correct hits.

Part B — Implement it in code [Python recommended]

In a .py script (reusable code with tests, not a notebook), implement these **from scratch** using only basic loops/arithmetic (no NumPy inside your functions), then verify each against NumPy on random inputs within 1e-9:

```
dot_product(a, b)
magnitude(v)            # sqrt(sum of squares)
cosine_similarity(a, b) # dot(a,b) / (magnitude(a) * magnitude(b))
mean(v)
variance(v)
```

> **Thread — the cosine-similarity seed**
>
> Flag **cosine_similarity** specifically. It is the *exact* operation used in Week 3 (embeddings) and Week 5 (RAG retrieval) to measure how similar two pieces of text are. You are building a foundation piece now; you'll watch it travel.

> **Hint — what "verify against NumPy within 1e-9" means**
>
> You're checking your hand-written function gives the same answer as the trusted library. NumPy has a built-in version of each of these (e.g. np.dot). For the same inputs, run *both* and check they agree. 1e-9 is just 0.000000001 — a tiny tolerance — because computers have tiny rounding differences, so you don't require *exactly* equal, just equal to nine decimal places. In code: abs(mine - numpys) < 1e-9.

Target

Mentor agrees the explainer reflects real understanding (not restated definitions); all functions match NumPy within 1e-9 across 100 random test cases.

Deliverable

A 2–3 page explainer (Markdown/PDF) + a .py module with the functions and passing comparison tests.

Task 1.3 — Take one messy dataset from raw to analysis-ready, reproducibly

●●● ~1 week

What you're doing

This is the week's main project, and it replaces what used to be two separate tasks. You take **one real dataset** and carry it the entire distance — inspect, clean, explore with charts, and engineer features — producing a pipeline that re-runs from scratch and a short findings note. This is exactly the shape of real data work: messy in, trustworthy out.

Dataset

Use a dataset with at least 8 columns and 500+ rows that has **real problems** in it. Good options: the Titanic dataset (seaborn.load_dataset('titanic')), Ames Housing, or any "dirty data" / "messy data" practice set from Kaggle. If your mentor provides a messy dataset, use that.

Tools & form

pandas, matplotlib/seaborn. Because cleaning must be **repeatable**, the pipeline lives in a .py script *or* a notebook that runs top-to-bottom with no manual steps. The exploratory charts may be in a notebook. Either way, re-running reproduces the clean output.

Phase 1 — Load & inspect

- Load into a DataFrame. Print .shape, .head(), .info(), .describe(). Write 2–3 sentences: what is one row? what are the columns?
- Use .isnull().sum() to map missing values per column.

Phase 2 — Clean (detect and fix each problem)

- **Duplicate rows** — find and remove (.duplicated(), .drop_duplicates()).
- **Inconsistent categories** — e.g. "USA", "U.S.", "us", "United States" → standardize to one value.
- **Broken / mixed types** — a numeric column stored as text, dates in mixed formats → convert to proper types.
- **Whitespace / casing** — " Male" vs "male" → trim and normalize.
- **Missing values** — for each affected column, choose and **justify** a strategy (drop, fill with mean/median, or fill with a category like "Unknown").

Phase 3 — Explore (at least 4 labeled charts)

- A histogram of a numeric column; a bar chart of a category; a correlation heatmap; and a relationship chart between two variables (e.g. an outcome rate by group).

> **Constraint:** every chart MUST have a title, axis labels, and a one-line takeaway written beneath it. A chart with no labels does not count.

Phase 4 — Engineer features & summarize

- Create 2–3 new features from existing columns; one line each on why it's useful (e.g. combine SibSp + Parch into family_size; extract a title from a name).
- Write 3–5 specific insights you can defend with the data (e.g. "Women survived at ~74% vs ~19% for men").

Show a before/after summary

```
Before:  5,000 rows | 320 duplicates | 412 nulls | 'price' stored as text
After:   4,680 rows |   0 duplicates |   0 nulls | 'price' as float, +2 new features
```

> **✨ Thread — data ethics & PII**
>
> Before you publish anything: check your dataset for **personal or sensitive information** (names, IDs, locations, health/financial details). Do not commit raw personal data to a public repo, and note in your findings whether any column would need protecting in a real system. In professional work — especially government or health data — this judgment is part of the job, not an afterthought.

Target

Zero duplicates, no unhandled nulls, all columns correctly typed, categories standardized; the pipeline re-runs cleanly from scratch; 4+ labeled charts; 2–3 justified features; 3–5 defensible insights; a clear before/after summary.

Deliverable

The cleaning pipeline (.py or top-to-bottom notebook) + an EDA notebook with labeled charts + the before/after summary + your findings note.

Rubric — Task 1.3

Not yet	Approaching	Meets (target)	Exceeds
Pipeline doesn't re-run cleanly, or major problems (dupes/nulls/types) remain.	Most cleaning done but something's unhandled, charts unlabeled, or no features engineered.	All four phases complete to target; pipeline reproducible; charts labeled; features justified; insights defensible.	Plus thoughtful feature engineering, an ethics/PII judgment, and analysis that surfaces a non-obvious finding.

Week 1 — Definition of Done

- A reproducible Python environment and a Git repo with clean history and no secrets.
- A math explainer showing real understanding + from-scratch functions verified against NumPy (including cosine_similarity).

- One messy dataset taken end-to-end into analysis-ready form, reproducibly, with labeled charts and defensible insights.

> **Pacing note for mentors**
>
> Don’t rush this week. A trainee new to Python may spend most of a day on setup and Git — that’s normal. The main project is where understanding compounds; prioritize comprehension over speed.

Week 2 — Classical Machine Learning

Week 2 is where you build your first real models. “Classical” ML means everything before deep learning — linear regression, decision trees, random forests. It’s the right starting point because the **workflow** you learn here (split → train → evaluate → improve) is identical for deep learning and LLMs later. Master it and neural networks become far less mysterious.

> **This week’s main project**
>
> One prediction problem, taken through a model’s **entire lifecycle** in a single project: establish a baseline, compare two algorithms, evaluate with the right metrics, then improve and validate with cross-validation — demonstrating overfitting along the way. This consolidates four old tasks into one realistic workflow, with **evaluation as the star**.

Main library: scikit-learn (sklearn). Modeling is exploratory, so a **notebook** is appropriate for the main project; any reusable helper goes in a .py file.

Task 2.1 — Explainer + prove overfitting with your own eyes

●● 2–3 days

What you’re doing

First write a short explainer of the core ideas, then run an experiment that makes overfitting **visible**. Reading about overfitting is forgettable; watching your model memorize noise is not.

Part A — Explainer (~1.5 pages, your own words)

- **Features vs. labels** — features (inputs, X) are what you know; the label (y) is what you predict.
- **Train / validation / test split** — why we split three ways and never let the model see test data during training (you don’t give students the exam while they study).
- **Overfitting vs. underfitting** — memorizing noise vs. too simple to capture the pattern.
- **Bias–variance trade-off** — one paragraph, plain language.

Part B — Overfitting demo (notebook)

- Take a small noisy dataset (e.g. a sine wave + noise). Fit polynomials of degree 1, 4, and 15. Plot each against the data — the degree-15 curve wiggles through every point: overfitting made visible.
- Plot training error vs. validation error as complexity rises. Training error keeps dropping while validation error starts climbing — the classic overfitting “U”.

> ✨ **Thread — the evaluation mindset begins here**
>
> This week plants the idea the whole program returns to: **a number that looks good can be lying to you**. Hold onto that instinct — it reappears in Weeks 3, 4, 5 and becomes a real harness in Week 6.

Target

Your plots clearly show the high-complexity model fitting noise and the train/validation curves diverging.

Deliverable

Explainer (Markdown/PDF) + overfitting_demo.ipynb with both plots and a short written interpretation.

Task 2.2 — Build, evaluate, and improve a model end-to-end

●●● ~1 week

What you're doing

Pick **one** prediction problem — regression (predict a number) or classification (predict a category) — and take a model through its whole life in a single coherent project. The point isn't the algorithm; it's that you can baseline, compare, evaluate honestly, improve, and validate.

Choose your track

Regression — e.g. California Housing (sklearn.datasets.fetch_california_housing), predict house value. Metrics: MAE, RMSE, R^2.

Classification — e.g. Titanic (predict survived) or breast cancer (sklearn.datasets.load_breast_cancer). Metrics: accuracy, precision, recall, F1, confusion matrix.

Do **one** track fully. (Ambitious trainees may do the second as stretch.)

Step 1 — Baseline first (always)

- Compute a dumb baseline: for regression, always predict the **mean**; for classification, always predict the **majority class**. Your real model must beat this clearly or something is wrong. This habit — never ship a model that can't beat trivial — is non-negotiable.

Step 2 — Prep & split

- Quick EDA (you know this from Week 1). Handle missing values; encode categoricals (pandas.get_dummies / OneHotEncoder); scale if needed. Split with train_test_split (e.g. 80/20). The model trains on training data only.

> **Hint — the jargon in that step, unpacked**
>
> - **Encode categoricals** — models only understand numbers, not words. So a "sex" column of "male"/"female" must become numbers (e.g. two 0/1 columns). pandas.get_dummies does this for you in one line.
> - **Scale features** — put columns on a comparable range so one big-numbered column (like income in the thousands) doesn't drown out a small one (like age). Some models need this; tree-based ones don't.
> - **train_test_split (80/20)** — randomly set aside 20% of rows as a "final exam" the model never sees while learning, so you can honestly test it on data it hasn't memorized.

Step 3 — Train & compare two algorithms

- A simple baseline model (LinearRegression or LogisticRegression) **and** a stronger tree-based one (RandomForest…). Always compare more than one — you learn why one wins.

Step 4 — Evaluate honestly (the heart of the week)

Regression: report MAE (average error size), RMSE (punishes big misses harder), R^2 (fraction of variance explained).

Classification: report accuracy **and why it lies** when classes are imbalanced, then precision ("when it says yes, how often right?"), recall ("of the real cases, how many found?"), F1, and a confusion matrix. Explain which of precision/recall matters more *for your problem* (e.g. for cancer screening, recall — a missed case is far worse than a false alarm).

Worked example — regression comparison

Model	MAE	RMSE	R^2
Mean baseline	0.91	1.15	0.00
Linear Regression	0.53	0.74	0.58
Random Forest	0.33	0.51	0.80

Step 5 — Improve & validate

- **Feature engineering** — create new features (e.g. Titanic: family_size from SibSp+Parch; title from name).
- **Hyperparameter tuning** — GridSearchCV or RandomizedSearchCV (e.g. tune n_estimators, max_depth).
- **Cross-validation** — report 5-fold cross_val_score, **not** a single split. A jump on one split can be luck; a jump on 5-fold CV is real.
- **Experiment log** — a table of what you changed and the cross-validated score.

Worked example — experiment log

Experiment	CV Score
Baseline model	0.78
+ family_size feature	0.80
+ title feature	0.82
+ tuned max_depth (GridSearch)	0.84

Target

Your better algorithm beats the dumb baseline by a clear margin; you improve on your own first score by a documented, cross-validated amount (e.g. +0.03); and you can explain in writing what moved the needle and why.

Deliverable

One notebook taking the problem end-to-end: baseline, two-algorithm comparison table, full metrics (with confusion matrix if classifying), the experiment log, and your written interpretation. Optionally save the final model with joblib.dump — a small but real production skill.

Rubric — Task 2.2

Not yet	Approaching	Meets (target)	Exceeds
No baseline, only one algorithm, or evaluation is just accuracy/one metric; no real improvement shown.	Two algorithms and metrics present, but no cross-validation, weak interpretation, or improvement is on a single lucky split.	Beats baseline; two algorithms compared; full appropriate metrics; cross-validated improvement over own baseline; clear written reasoning.	Plus thoughtful feature engineering, a correct precision/recall (or error-cost) argument for the domain, and honest analysis of what didn't help.

Week 2 — Definition of Done

- Explain supervised learning, the train/val/test split, and overfitting — and demonstrate overfitting with a chart.
- Take one model through baseline → comparison → proper evaluation → improvement → cross-validation in a single coherent project.
- Use scikit-learn fluently for the full loop, and reason about why a metric can mislead.

> **Pacing note for mentors**
>
> The single most important outcome is that they deeply understand evaluation — why accuracy isn't enough, why cross-validation beats one split. The algorithms are secondary and easily swapped. Expect looping (realizing mid-improvement that their evaluation was off); that looping is the learning.

Week 3 — Deep Learning

Week 3 is the biggest conceptual jump in the program. You move from classical ML to **neural networks** — the technology behind modern AI, including the LLMs of Weeks 4–6. The week has two linked halves: build a tiny neural net **by hand** to truly understand how it learns, then use a real framework to build bigger ones on real data, and finish at **embeddings** — the bridge to LLMs.

> **This week's main project**
>
> Two builds under one arc, "from neuron to representation." **(A)** a neural network from scratch in NumPy that solves XOR — the keystone where backprop stops being magic. **(B)** a PyTorch image classifier on real data with a transfer-learning comparison, closing with a small **embeddings** semantic search that reuses your Week 1 cosine similarity.

Libraries: NumPy (from-scratch — no DL library allowed there, on purpose), PyTorch (torch, torchvision), matplotlib. CPU is fine; no GPU needed. **Install:** pip install torch torchvision matplotlib

Task 3.1 — Build a neural net in NumPy and train it to solve XOR

●●● ~1 week [Python required]

What you're doing

Build a small neural network using **only NumPy** — implementing the math yourself — and train it to solve **XOR**. This is the single most important task of the week. It's hard; budget the time and expect to get stuck. That's the point: almost every engineer who truly "gets" deep learning built one by hand once.

Background (learn before coding)

Watch 3Blue1Brown's *Neural Networks* series (4 short videos) and read Michael Nielsen's free book, ch. 1. Know these terms: a **neuron** computes activation(weighted sum + bias); a **layer** is many neurons; **forward pass** feeds input through to a prediction; **loss** measures wrongness; **backpropagation** uses the chain rule to find how each weight contributed to the error; **gradient descent** nudges weights to reduce loss.

Why XOR?

A network with no hidden layer mathematically cannot solve XOR. If yours learns it, your hidden layer and backprop are correct. It's the standard "did I build this right?" test:

```
Input    | Output
(0, 0)   |   0
(0, 1)   |   1
(1, 0)   |   1
(1, 1)   |   0
```

Steps

1. Architecture: 2 inputs → 1 hidden layer (start with 4 neurons) → 1 output. Initialize weights to small random numbers, plus biases.
2. Forward pass with the sigmoid activation: 1/(1+e^-x).
3. Loss: mean squared error between prediction and label.
4. Backward pass: compute gradients via the chain rule. Hint — the derivative of sigmoid is sigmoid(x)*(1-sigmoid(x)). This is the hardest part; go slowly.
5. Update: weight = weight - learning_rate * gradient (start learning_rate = 0.1).
6. Train for several thousand epochs on the 4 examples; print loss every 500 — it should steadily fall. Plot the loss curve.

Hint — how to actually approach this without drowning

- **An "epoch"** is one full pass over your training data. Several thousand passes over just 4 examples is normal here — it's how the network slowly tunes its weights.
- **Build it in this order, testing as you go:** (1) get the forward pass working and just print a prediction — ignore learning for now; (2) confirm your loss number computes; (3) *then* add the backward pass. Don't write all of it and hope — verify each piece runs before adding the next.
- **If the loss won't drop,** 9 times out of 10 it's the backward pass or a shape mismatch. Print the shape of every array (print(x.shape)) and check the math by hand on one example. This slow, deliberate checking *is* the skill — not a sign you're failing.

Thread — debugging is a first-class skill

- Forgetting the bias term → network can't learn.
- Wrong matrix dimensions → shape errors. Print shapes at every step to debug — this is the real skill being trained.
- Learning rate too high → loss explodes/oscillates; too low → loss barely moves.
- Wrong derivative in backprop → loss won't drop.

Target

After training, outputs are close to [0, 1, 1, 0] (e.g. <0.1 for the 0s, >0.9 for the 1s) — ≥95% accuracy — with a loss curve that clearly trends down. You can explain, in your own words, how backprop works.

Deliverable

nn_from_scratch.py + a saved loss-curve plot + a top-of-file comment block explaining backprop in your words, written after you finish.

Concept companion (written, mentor-graded)

Answer in your own words; your mentor will discuss these:

- What does an activation function do, and why must it be non-linear? (What could the network represent if every layer were purely linear?)
- In plain language, what is backpropagation actually doing?
- What is the learning rate? What goes wrong if it's far too high, or far too low?
- Training loss keeps dropping but validation loss starts rising. What is happening, and what's it called? ✨ (the evaluation thread, again)

Rubric — Task 3.1

Not yet	Approaching	Meets (target)	Exceeds
Network doesn't learn XOR (loss flat or exploding), or a DL framework was used.	Solves XOR but the trainee can't explain backprop, or relied heavily on copied code.	Solves XOR ≥95% with a falling loss curve, written in pure NumPy, and can explain backprop and the concept questions.	Plus experiments: varying hidden size / learning rate, or extending to a second problem, with insight.

Task 3.2 — PyTorch image classifier + transfer learning + embeddings search

●●● ~1 week [Python recommended]

What you're doing

Now that you understand the mechanics by hand, use **PyTorch** to build real models efficiently, see why transfer learning wins, and finish at embeddings — the bridge to everything in Weeks 4–6.

Part 1 — Train a classifier with PyTorch

Start on MNIST (handwritten digits, built into torchvision) to learn the workflow, then move to **CIFAR-10** (color images, harder) for the real build. Implement the standard pieces:

- Data loading in batches (DataLoader), with tensors normalized; on CIFAR-10 add data augmentation (random flip/crop) to the training set.
- A model subclassing nn.Module. For images, build a small CNN: a few blocks of Conv2d → ReLU → MaxPool2d (increasing channels, e.g. 32→64→128), then flatten → Linear → outputs. Understand what convolution (local pattern detection) and pooling (shrink, keep strongest signal) do.
- Training loop: forward → CrossEntropyLoss → loss.backward() (PyTorch does the backprop you hand-coded in 3.1!) → Adam optimizer step → zero gradients. Validate each epoch with torch.no_grad(); plot loss and accuracy curves.
- **Add one regularization technique** (Dropout, weight decay, or early stopping); train with and without it and report the effect on the train/test accuracy gap.

Part 2 — Compare with transfer learning

Take a pretrained resnet18(weights='DEFAULT'), replace its final layer for 10 classes, and fine-tune on CIFAR-10. Compare accuracy **and** epochs-to-get-there against your from-scratch CNN. Write up why the pretrained model wins (it already learned general visual features from millions of images — you're standing on that work). This "don't start from zero" lesson is how real image AI is built.

Targets

- From-scratch CNN: ≥70% test accuracy on CIFAR-10 (80%+ is a good stretch).
- Transfer-learning model: ≥85%, reached in fewer epochs.

Part 3 — Embeddings semantic search (the bridge)

Build a tiny "semantic search": given a query, find the most **meaningfully similar** texts from a collection, even with no shared words. This is a miniature of Week 5's retrieval.

- **Library:** sentence-transformers (free, local, no API key). Model all-MiniLM-L6-v2. Install: pip install sentence-transformers
- Create 20–50 short, varied texts. Embed them all. For a query, embed it and compute cosine similarity to every text; return the top-k by score.

✨ Thread — cosine similarity returns

The “closeness” you measure here is the **exact cosine_similarity** you hand-coded in Week 1 (Task 1.2). Same arithmetic, now operating on vectors that carry meaning. You may call the library, but recognize what it’s doing under the hood. **And it returns once more in Week 5**: the vector database in your RAG system is doing precisely this similarity search — the loop you’re writing here — just scaled up and made fast. Build it by hand now and the database will hold no mystery later.

The critical test — prove it’s semantic, not keyword

```
Query: "I forgot my login credentials"
Top results:
  0.71  How do I reset my password?      <- zero shared keywords, ranked #1
  0.42  Where can I update my email?
  0.19  What are your business hours?
```

Target (Part 3)

For 5 hand-written queries including at least one meaning-not-keywords case, the top result is sensible in ≥4 — and the non-keyword case works.

Deliverable

cifar_cnn.py (your CNN) + the transfer-learning comparison (script/notebook) + saved loss/accuracy curves + semantic_search.py with demo output + a short written explanation covering convolution/pooling, your preprocessing, why transfer learning won, and what embeddings are.

Rubric — Task 3.2

Not yet	Approaching	Meets (target)	Exceeds
CNN well below target or doesn’t train; no transfer-learning comparison; embeddings search missing or keyword-only.	CNN trains but misses target, or the transfer/embeddings pieces are incomplete or unexplained.	CNN ≥70%, transfer model ≥85% in fewer epochs, embeddings search passes the semantic test; all explained.	CNN pushed toward 80%+, thoughtful augmentation/regularization analysis, and a crisp written account of why transfer learning wins.

Stretch (optional)

A text sentiment classifier on IMDb (clean → TF-IDF → logistic regression, target >85%) to touch the other major unstructured data type — only if Week 3 is going smoothly.

Week 3 — Definition of Done

- Build a neural network from scratch (forward pass, backprop, gradient descent) and have it solve XOR — and explain how it works.
- Use PyTorch to train an image classifier to target accuracy, with batching, validation, and a regularization technique.

- Explain why transfer learning outperforms training from scratch, and demonstrate it.
- Understand and use embeddings for semantic similarity — the bridge to LLMs and RAG.

> **Pacing note for mentors**
>
> This is the hardest week so far, and the from-scratch network (3.1) is where trainees struggle most — getting backprop right is genuinely difficult. Budget real time and expect to help debug matrix shapes and gradient math. If someone is badly stuck, pair-program the backward pass rather than let them lose three days. A trainee who fights through 3.1 understands deep learning in a way framework-only learners never quite do.

Week 4 — Large Language Models, Applied

Weeks 1–3 built your foundation. Week 4 is where you start doing what applied AI engineers actually do day-to-day: **building on top of LLMs** (Claude, GPT, Gemini, Llama). You'll first understand — at a working level — what a transformer is and how text becomes tokens, then spend the week building.

This week's main project

One assistant that grows across the week and forces every core LLM skill to cooperate: real **API calls** with secrets handled, **prompt iteration** until reliable, **structured output the correct modern way** (the API's native tool-use, not coaxing JSON from prose), **tool calling**, and finally **memory + persona** in a multi-turn chatbot.

You do NOT need to train an LLM

Training one costs millions. Applied AI engineering is about **using** powerful hosted models well — that's the job. Throughout, "the model" means the hosted LLM you call via API, not one you trained.

APIs / libraries (free tiers or fully local): Anthropic Claude or OpenAI SDK (free trial credits), or **Ollama** for a free local option (ollama pull llama3); tiktoken for tokenization; python-dotenv for secrets; pydantic for validation.

Thread — professional hygiene: API keys

Never hard-code an API key or commit it to Git. Store it in a .env file, load with python-dotenv, and confirm .env is in .gitignore. A leaked key can cost real money. This rule is absolute, all week and after.

Task 4.1 — Transformers & attention explained + tokenization hands-on

●● 2–3 days

What you're doing

Two short pieces so LLMs aren't a black box: explain the core idea in your own words, then explore tokenization in code. This directly explains the costs and limits you'll hit all week.

Part A — Transformers & attention (~1 page)

Learn from Jay Alammar's *The Illustrated Transformer* and 3Blue1Brown's transformer videos. Then explain, in plain language:

- **What transformers solved** — earlier models read word-by-word and struggled with long-range links; transformers look at all words at once.
- **Attention** — for each word, the model learns which other words to focus on. In “The animal didn’t cross the street because it was too tired,” attention helps “it” refer to the animal. Explain in your own words.
- **The context window** — because it processes a whole context at once, there’s a maximum it can consider, which is why long inputs get cut off and cost scales with length.

Part B — Tokenization, hands-on

LLMs see **tokens** (chunks of text), not letters or words. You’re billed per token and the context window is measured in tokens. Install tiktoken and write a script that, for any text, shows the tokens and the count. Then record what you find:

- A simple English sentence: tokens vs. words.
- A long rare word (“antidisestablishmentarianism”): notice it splits into many tokens.
- **English vs. Urdu** (or another non-Latin script) for the same sentence: notice non-English often uses far more tokens — a real cost implication for multilingual products.
- Cost connection: look up a model’s price per 1K tokens and compute the cost of sending a 2,000-word document.

Expected output

```
Text: "AI is transforming software."
Tokens: ['AI', ' is', ' transforming', ' software', '.']
Token count: 5   (Word count: 4)
```

Target

A ~1-page explainer your mentor agrees is non-hand-wavy + a working tokenizer script with written observations (English vs. Urdu, rare words, the cost calc).

Deliverable

explainer (Markdown/PDF) + tokenizer_explorer.py + observations.

Task 4.2 — API → prompts → structured output → tools → a chatbot with memory

●●● ~1 week

What you're doing

One build that you grow in five stages. Each stage adds a capability and reuses the last, so by the end you have a real, multi-turn assistant that prompts well, returns clean structured data, uses tools, and remembers the conversation. Deliver **.py scripts/modules**, not notebooks.

Stage 1 — First API call, secrets done right

- Put your key in .env (gitignored), load via python-dotenv. Send a system message (sets behavior) + a user message; parse and print just the text content (inspect the response object deliberately — don't blind-copy).

> **Hint — system message vs user message**
>
> The **system message** is the standing instruction that shapes how the model behaves for the whole conversation — like "You are a concise assistant who always answers in plain English." The **user message** is the actual question or request for this turn. Think of the system message as the job description and the user message as today's specific task.

- **Experiment with temperature**: send the same prompt at temperature 0 and 1 a few times; record the difference (0 = focused/near-deterministic, higher = more varied).

Stage 2 — Prompt iteration study

Take a task a naive prompt does badly (e.g. extract names/dates/amounts from messy emails; classify support tickets; summarize to exactly 3 bullets). Improve the prompt one technique at a time, **measuring each** on the same 10 test inputs:

- Baseline (naive one-liner) → clear specific instructions → 2–3 few-shot examples → role/system framing → step-by-step for reasoning. Keep a results table.

> **Expected log — ✨ the evaluation thread, applied to prompts**

```
Prompt version          | Correct (out of 10)
------------------------|--------------------
Naive one-liner         | 4
+ clear instructions    | 6
+ few-shot examples     | 9
+ step-by-step          | 10
```

Stage-2 target: the final prompt is correct on ≥9/10 inputs where the naive prompt was unreliable.

Stage 3 — Structured output, the correct modern way

Real apps need to **parse** the model's output into data. The outdated habit is begging the model for JSON in prose and hoping. The **correct modern approach** is to use the API's native structured-output mechanism — the same **tool-use / function schema** the providers give you — so the model returns data shaped to a schema you define. Do it this way:

- Define a target schema (e.g. for a support ticket: category, priority, summary, sentiment with allowed values).
- Pass it to the API as a **tool/structured-output schema** (Anthropic and OpenAI both support this — read their docs). Then validate the result with **pydantic**. Handle failure gracefully (retry, or repair) so one bad response never crashes the app.

> **Why this matters (don't skip the reasoning)**
>
> Prompt-coaxed JSON works until it doesn't — a stray "Here's your JSON:" or a markdown fence breaks json.loads(). Native structured output is how production systems get reliable data out of an LLM. Learning the prose-coaxing habit now would be learning to do it the outdated way; this stage exists to teach the right one.

Stage-3 target: valid, schema-conforming output on ≥19/20 varied inputs, with graceful failure handling.

Stage 4 — Tool calling

Let the model decide when to call your functions. **The model chooses and supplies arguments; your code executes and feeds the result back; the model answers.** This is the foundation of agents (Week 5).

- Define 2 real tools, e.g. calculator(expression) and get_current_time(timezone) (or a fake get_weather(city) with canned data).
- Describe them to the API in its tools schema. Implement the loop: send question + tools → if the model requests a tool, run it → send the result back → model gives the final answer.

> **Hint — the one idea people get wrong about tool calling**
>
> The model **does not run your function**. It can't — it only outputs text. What actually happens: the model *replies* "I'd like to call calculator with 4892*17." **Your code** sees that, runs the real function, and sends the answer back. The model then uses that answer to write its final reply. So the model **decides** and **you execute**. If you remember nothing else about tool calling, remember that split — it's the whole basis of agents next week.
>
> **Test that it chooses correctly**

```
"What is 4,892 × 17?"        -> should call the calculator
"What time is it in Tokyo?"   -> should call the time function
"Who wrote Hamlet?"           -> should answer directly, NO tool
```

Stage-4 target: across 10 mixed queries, the model chooses correctly ≥9 times — including correctly NOT calling a tool when unnecessary (don't over-call).

Stage 5 — Wrap it in a multi-turn chatbot

Combine everything into a chatbot with real **memory** and a consistent **persona**. LLM APIs are stateless — each call knows nothing of the last — so **you** keep the full message list and send it every turn. The persona is a well-written system message kept at the top.

```
[
  {"role": "system",    "content": "You are a friendly travel assistant..."},
  {"role": "user",      "content": "I want to visit Japan"},
  {"role": "assistant", "content": "Great choice! When are you..."},
  {"role": "user",      "content": "In spring"}   <- model sees all above
]
```

- Loop: read input → append → send whole list → get reply → append → show. Handle edge cases: empty input (re-prompt, don't crash); a long conversation nearing the context limit (trim or summarize older turns); an exit command (quit gracefully).
- Interface: a CLI loop is completely acceptable. A minimal Streamlit UI is optional — don't over-invest; the logic is what matters.

Stage-5 target: a 6+ turn conversation where the bot uses information from turn 1 when answering at turn 5 (memory works), stays in persona, and survives the edge cases.

Overall target (the whole project)

All five stage targets met, integrated into one assistant: it prompts reliably, returns valid structured output via the native mechanism, uses tools correctly (including abstaining), and holds a multi-turn conversation in persona.

Deliverable

A small project (e.g. first_llm_call.py, prompt_study + results table, structured_output.py, tool_calling.py, chatbot.py, optional Streamlit file) + transcripts demonstrating each capability, especially memory and correct tool choice.

Rubric — Task 4.2

Not yet	Approaching	Meets (target)	Exceeds
A capability is missing or broken; secrets hard-coded; or JSON coaxed from prose rather than native structured output.	Most stages work but one misses target (e.g. over-calls tools, memory flaky), or the trainee can't explain the model-decides/you-execute loop.	All five stage targets met and integrated; native structured output used; secrets clean; trainee can explain each piece.	Plus polish: robust failure handling, thoughtful context-window management, or a genuinely useful end product.

Week 4 — Definition of Done

- Explain transformers/attention and the context window; reason about token cost and limits hands-on.
- Make LLM API calls with proper secret management; understand system/user messages and temperature.
- Do prompt engineering systematically with measured improvement.
- Get reliable structured output via the API's native mechanism, with validation and failure handling.
- Implement tool calling (model decides, your code executes), including correct abstention.
- Build a multi-turn chatbot with real memory and a consistent persona.

> **Pacing note for mentors**
>
> A relief after Week 3 — more building, less heavy theory — but the concepts in 4.1 and the tool-calling loop are where understanding (not just working code) matters most, because Week 5's agents depend on them entirely. Probe that they truly get model-decides/you-execute. Budget for API/key setup friction on day one; if cost is a concern, push everyone to Ollama — the learning is identical.

Week 5 — RAG & Agentic Development

The most job-relevant week. The two architectures here — **RAG** (Retrieval-Augmented Generation) and **agents** — are what most real-world AI products are made of. A huge share of applied AI work is some variation of "make an LLM answer accurately from *our* documents" (RAG) or "make an LLM accomplish a multi-step task with tools" (agents). Everything from Weeks 3–4 converges here.

The problem RAG solves

LLMs don't know your private documents and they hallucinate when asked beyond their training. RAG fixes both: instead of hoping the model knows, you **retrieve** the relevant text from your own documents and hand it to the model to answer from. This is how "chat with your PDF," internal knowledge assistants, and tools like Wonit.ai work.

This week's main project

Two builds. **(A)** a complete RAG "chat with your documents" system — retrieval pipeline folded in as phase 1, with **grounding and honest refusal** as the hard, important part. **(B)** a from-scratch **Reason → Act → Observe** agent that chains 2–3 tools to finish a multi-step task, with a step limit.

Libraries (all free/open-source): ChromaDB (local vector DB, zero setup; Qdrant — what Wonit.ai uses — is a worthwhile alternative to see); sentence-transformers (all-MiniLM-L6-v2); an LLM (Claude/OpenAI/OpenRouter, or Ollama locally); pypdf or pdfplumber for PDFs.

Build it raw first

Build the RAG loop and the agent **without an orchestration framework** so you understand the moving parts. A short reading note at the end covers what a framework would wrap — read, not built. Frameworks debugged through a black box you don't understand are a classic trap.

Task 5.1 — A retrieval pipeline + a grounded, citing, honestly-refusing RAG app

●●● ~1 week

What you're doing

Build the full RAG loop — the flagship application of the week. A user asks a question; your system retrieves relevant context from the documents and has the LLM answer using **only** that context, citing sources — and saying "I don't know" when the answer isn't there.

Phase 1 — Ingestion (done once)

- Load documents (text/markdown or PDFs via pypdf/pdfplumber). Chunk them into ~500-character pieces with a small overlap (~50 chars) so an idea on a boundary still appears whole in one chunk.

> **Hint — chunking, overlap, and "top-k" in plain words**
>
> - **Chunking** — cutting a long document into bite-sized pieces. You can't hand the model a whole 50-page PDF for every question, and searching tiny pieces is far more precise than searching whole documents.
> - **Overlap** — letting consecutive chunks share a little text at the edges. If an important sentence sits right where you cut, overlap makes sure it still appears *whole* in at least one chunk instead of being sliced in two.
> - **Top-k** — "give me the k most similar chunks." If k=3, you retrieve the 3 closest pieces to the question. It's just how many results you pull back.

- Embed each chunk (sentence-transformers, all-MiniLM-L6-v2) and store text + vector in ChromaDB. ✨ the embeddings + cosine similarity are the same idea from Weeks 1 and 3.

> **✨ Thread — you already built this by hand**
>
> In Week 3 (Task 3.2, Part 3) you built a semantic search by computing cosine similarity over an in-memory list of texts — looping through every item to find the closest. **That is exactly what a vector database does under the hood.** ChromaDB just does that similarity search for you, persistently and fast, over far more chunks than a hand-written loop could handle. You are not learning a new idea here — you are scaling up the one you already understand. (Qdrant, which *Wonit.ai* uses, is the same concept at production grade; worth seeing, but ChromaDB is all you need to learn the pattern.)

Phase 2 — Retrieval (per query)

- Embed the query, ask the vector DB for the top-k most similar chunks (e.g. k=3), return them with scores.
- **Chunk-size experiment**: try ~300 vs ~800 characters and note the effect on retrieval quality (too big = irrelevant text included; too small = lost context).

Phase 3 — Generation, grounded

Build a prompt that includes the retrieved chunks as **context** plus the question, and instructs the model to answer using **only** that context, cite the chunk numbers used, and say it doesn't know if the answer isn't present.

```
System: Answer using ONLY the context below. If the answer is not in the
context, say "I don't have that information in the documents." Cite the
source chunk numbers you used.

Context:
[1] {chunk 1 text}
```

```
[2] {chunk 2 text}
Question: {user question}
```

> **The hard part — refuse to hallucinate**
>
> Handling the “not in documents” case is what separates a real RAG system from a demo. If retrieval returns nothing relevant (low scores) or the answer isn’t there, the system must say so rather than invent. Test this deliberately.

Test set — ✨ the evaluation thread, applied to RAG

Build 10 questions: 8 answerable from your documents (must answer correctly with sources) and 2 not in the documents (must correctly decline, not fabricate).

Target

On the 10 questions, it answers the 8 correctly with sources and correctly declines the 2 — grounded answers, no hallucination on the out-of-scope ones.

Deliverable

retrieval.py (ingest + retrieve) with the chunk-size note + rag_app.py (+ optional Streamlit/CLI UI) + your 10-question test results (question, answer, sources, pass/fail) + a transcript.

Rubric — Task 5.1

Not yet	Approaching	Meets (target)	Exceeds
Retrieval doesn’t return relevant chunks, or the app hallucinates instead of grounding/citing.	Answers from documents but doesn’t cite, or fails the out-of-scope refusal, or no chunk-size analysis.	8/8 answered with sources, 2/2 correctly declined; chunking understood; retrieval solid.	Plus careful chunk-size tuning, clean citations, and robust refusal across tricky edge cases.

Task 5.2 — Build a Reason → Act → Observe agent from scratch

●●● ~1 week

What you're doing

In Week 4, tool calling was *one* round. An **agent** does it in a **loop**, autonomously, to handle multi-step tasks. You give it a goal and tools; it decides step by step what to do, uses tools, observes results, and continues until done.

The classic loop

```
Goal: "Find the population of France and Germany, then the combined total."

Step 1  Reason: "I need France's population." -> Act: search("France population")
        -> Observe: "68 million"
Step 2  Reason: "Now Germany's." -> Act: search("Germany population")
        -> Observe: "84 million"
Step 3  Reason: "Add them." -> Act: calculator("68000000 + 84000000")
        -> Observe: "152000000"
Step 4  Reason: "I have everything." -> Final answer: "~152 million."
```

Steps

6. Define 2–3 tools, e.g. search(query) (real API or canned facts for testing), calculator(expression), read_file(path).
7. Implement the loop: give the LLM the goal + tools + "work step by step"; the model returns either a tool call (your code runs it, feeds the result back) or a final answer; loop until final.
8. Make reasoning visible: print each Reason → Act → Observe step so you (and your mentor) can see it think. Essential for debugging agents.
9. Add a max-steps safeguard (e.g. stop after 10 loops) so a confused agent can't loop forever and burn money.

Target

The agent completes a task needing ≥3 chained tool calls end-to-end on 3 of 4 trials, with reasoning visible and the step limit in place.

Deliverable

agent.py with the tools and loop + a logged run showing the full Reason → Act → Observe trace for a multi-step task.

Reading note (not built)

In 3–5 sentences, describe what an orchestration framework (LangChain / LlamaIndex) would wrap for you here — the loop, tool schemas, memory — and one reason you'd still want to understand the raw version

underneath. This keeps your fundamentals intact while making you aware of the real-world tooling.

Rubric — Task 5.2

Not yet	Approaching	Meets (target)	Exceeds
Agent doesn't chain tools, loops uncontrollably, or has no visible reasoning / no step limit.	Completes simple cases but unreliable on ≥3-step chains, or reasoning/safeguard incomplete.	Completes a ≥3-tool task 3/4 trials with visible reasoning and a step limit; reading note done.	Plus graceful handling of tool failures or a genuinely useful multi-step task beyond the toy example.

Week 5 — Definition of Done

- Build a retrieval pipeline: chunk, embed, store, and retrieve relevant chunks for a query.
- Build a complete RAG system that answers grounded in documents, cites sources, and refuses to hallucinate when the answer isn't there.
- Understand and build a Reason → Act → Observe agent that chains tools for multi-step tasks, with visible reasoning and a step limit.
- Explain how these map to real products — and what a framework would wrap, without depending on one.

> **Pacing note for mentors**
>
> RAG (5.1) is the highest-value skill in the program for most applied roles. Expect the "refuse to hallucinate" part to be where they learn the most and struggle — getting grounding and "I don't know" right is subtle. For the agent (5.2), insist they build it from scratch; the reading note is enough framework exposure. If the week runs short, a slightly rough agent is an acceptable trade to keep RAG solid.

Week 6 — Production, Evaluation & Capstone

The final week is the difference between a *demo* and a *real system*, proven in one capstone. Anyone can get an LLM script working once on a good input. Production AI engineering is making it work **reliably, measurably, and affordably** — and knowing when it's failing.

> **This week's main project — a different shape**
>
> This week there are no separate pre-tasks. Evaluation and production hardening are **required ingredients of the capstone**, built in from the start rather than taught abstractly and bolted on. You'll learn each just-in-time as you build the one project that proves the whole program.

Tools (free / free-tier): your LLM access from Weeks 4–5 (OpenRouter gives many models via one key; Ollama is free/offline); pytest for the eval harness; python-dotenv and pydantic; a dict / diskcache / functools.lru_cache for caching; tenacity for retries; Python's logging.

Task 6.1 — An end-to-end AI application with evaluation and production basics baked in

●●● ~1 week

What you're doing

Design and build one complete AI application of your choosing that pulls together the whole program. A focused, working, well-built project beats an ambitious broken one — scope it realistically. The two skills that used to be separate tasks, **evaluation** and **production hardening**, are defined below and are required parts of *this* build.

Required ingredients — your project must include all five

10. An LLM at its core (Claude / OpenAI / OpenRouter / Ollama).
11. Retrieval (RAG) OR an agentic component — it answers from a document/data source, or accomplishes multi-step tasks with tools. (Both is a strong stretch.)
12. Real data handling — actual documents, a dataset, or an API, not hardcoded strings.
13. Evaluation — a small harness showing quality is measured, not assumed (defined below).
14. Production basics — at minimum error handling, secrets management, and caching (defined below).

Ingredient 4, defined — the evaluation harness ✨

This is the evaluation thread reaching its destination. Build a repeatable test suite that scores your system on known cases so you can measure quality with a single number and catch regressions.

- A test set of at least 15 cases. Each is an input + what a good output looks like. Include hard cases and some that should be refused (out-of-scope), so you test failure behavior too.

- **Rule-based checks** (fast, deterministic, free) — e.g. does the answer contain the expected key fact? Did it cite sources? Did it correctly refuse?
- **LLM-as-judge** for qualitative things rules can't catch — send question + answer to an LLM and ask it to score 1–5 with a reason. Spot-check the judge's grades yourself; judges are useful but not infallible.

> **Hint — what "LLM-as-judge" means**
>
> Some qualities can't be checked with a simple keyword rule — like "is this answer well-written and on-topic?" So you use a *second* LLM call purely as a grader: you show it the question and your system's answer and ask, "Score this 1–5 and explain why." It's an automated stand-in for a human reviewer. Because the judge can be wrong too, you read a handful of its grades yourself to make sure they're sensible — that's the "spot-check."

- **Prove it catches regressions**: deliberately break something (swap to a much smaller model, or break retrieval), re-run, and show the score drop. This proves the harness works.

Expected output

```
EVAL RESULTS
Rule-based:  13/15 passed
LLM-judge avg relevance: 4.3/5
Failures:
  - Case 7 ("warranty length"): missing key fact "2 years"
  - Case 12 (out-of-scope): hallucinated instead of refusing
```

Ingredient 5, defined — production basics

- **Error handling & retries** — wrap API calls so a transient failure retries with exponential backoff (1s, 2s, 4s). The tenacity library does this cleanly; the same logic covers rate-limit (HTTP 429) backoff.

> **Hint — "exponential backoff," minus the jargon**
>
> If an API call fails (a network blip, or you're sending too fast), don't give up and don't hammer it again instantly. **Wait, then retry — and wait longer each time**: 1 second, then 2, then 4. The growing pause gives the service time to recover and stops you from making a busy server worse. "Exponential" just means the wait doubles each round. The tenacity library adds this with a single line above your function, or you can write it with a short loop and time.sleep().

- **Caching with a measured win** — if the same input is seen twice, don't pay twice. Add a cache (dict / diskcache) and **measure** the saving (e.g. "cached responses return in 2ms vs 1,400ms"). "I added caching" isn't enough — show the before/after.

- **Logging** — basic logging (Python's logging) of requests, errors, cache hits. You can't debug production without logs. (This is the fundamental version of "observability" — no heavy tooling needed.)
- **Secrets** — all keys from environment / .env, nothing hard-coded, .env gitignored. Verify it. ✨ professional hygiene, one last time.

Example projects (pick one or bring your own)

- Document Q&A assistant — upload docs/papers/a textbook, ask questions, get sourced answers. (RAG)
- Research agent — give it a topic; it searches, gathers, and writes a short structured brief using tools. (Agent)
- Support-ticket triager — ingest tickets, classify + prioritize, draft a reply grounded in a help-doc knowledge base. (RAG + structured output)
- Data-analysis agent — give it a CSV; it answers questions by writing and running queries via tools. (Agent + data)

Process

7. Scope & design first — a half-page: what it does, who it's for, the architecture (components, model, data source), and what you're explicitly NOT building (cut scope honestly).
8. Build it, applying eval and production practices from the start — don't bolt them on at the end.
9. Evaluate it with your test set.
10. Demo + write-up — a short live demo (it must run end-to-end on a fresh input) and a written breakdown: architecture, key decisions and why, what you'd improve, and honest current limitations.

> ✨ **Thread — data ethics, at the finish**
>
> If your capstone touches real or sensitive data (especially anything resembling personal, government, or health records), state how you handle it: what's stored, what's sent to a third-party API, and what you'd need to protect in a production deployment. This judgment is part of a complete engineer's work.

Target

A working application that runs end-to-end on a fresh input during the demo, hits all five required ingredients, and for which you can answer "why did you build it this way?" for each major component.

Deliverable

The complete project in your GitHub repo with a clear README (setup + run) + the architecture write-up + the eval results + an honest limitations / next-steps list.

Rubric — Task 6.1 (the capstone)

Not yet	Approaching	Meets (target)	Exceeds
Doesn't run end-to-end, or is missing a required ingredient (no eval, no production basics, or no real data).	Runs but a required ingredient is thin (eval trivial, no measured caching win, or shaky on a fresh input).	Runs end-to-end on a fresh input; all five ingredients present and real; trainee explains every major decision.	Production-minded throughout: regression-catching eval, measured performance wins, clean architecture, and honest, insightful limitations.

Week 6 — Definition of Done

- Build an evaluation harness (rule-based + LLM-as-judge) that scores a system and catches regressions.
- Harden an app: retries with backoff, caching with a measured win, logging, and proper secrets.
- Design, build, evaluate, and demo a complete end-to-end AI application combining an LLM, retrieval or an agent, real data, evaluation, and production basics.
- Explain architectural decisions and limitations honestly.

> **Pacing note for mentors**
>
> The capstone is the centerpiece — protect time for it. The common failure is over-scoping into an impressive-sounding but broken project; push hard toward "small and actually working." Evaluation and hardening are required ingredients precisely so they get built in rather than skipped. The final "why did you build it this way?" discussion is the truest test of whether the six weeks landed.

After six weeks

A trainee who completes all six weeks should have become a beginner AI Engineer with strong fundamentals and understanding of AI in general.

This makes a foundation, not mastery in AI. A trainee who finishes can contribute to production AI work under guidance and keep deepening on their own, which is exactly the goal. The advanced topics deliberately left out (orchestration frameworks, re-ranking, multi-agent systems, MCP, and the rest) are a short step from here precisely *because* the fundamentals are solid.

Appendix — the assessment framework

Every main project is graded on the same four levels so review is consistent across trainees and mentors.

Not yet	Approaching	Meets (target)	Exceeds
Required pieces missing or the result doesn't run / doesn't meet the basic goal. Not ready to advance.	Runs and is close, but misses the numeric target, skips a required piece, or the trainee can't yet explain it.	Hits the stated Target, includes every required piece, and the trainee can explain what and why. This is the bar to advance.	Clearly beyond target — deeper analysis, cleaner code, extra rigor, or insight the task didn't demand.

How a mentor runs a weekly review

15. Trainee walks through their main-project deliverable and demos it running.
16. Mentor checks the Target line and the Deliverable list for completeness.
17. Mentor asks 2–3 "why" questions — "Meets" requires understanding, not just a passing number.
18. Mentor records the level for the main project. A week is signed off when the main project reaches Meets. Stretch items are noted but never required.

> **The single most important grading principle**
>
> A trainee who can **explain why** is worth more than one who only produced output. Where a number and an explanation disagree, trust the explanation: a passing metric the trainee can't account for is *Approaching*, not *Meets*.